Lesson 4

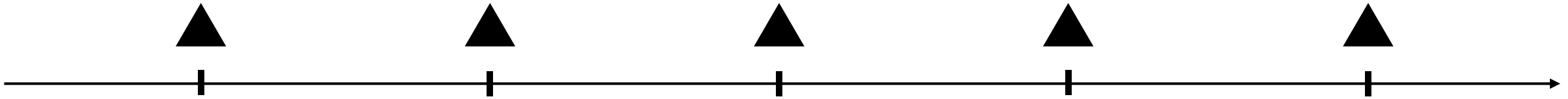
DAX Essentials: Foundations of Data Analysis Expressions

Unlock the Power of Calculated Measures & Columns

March 2025

Power BI Desktop workflow.

From Data to Insights



Connect to data sources

Import data from various sources like Excel, SQL Server, cloud services, or APIs.

Transform and clean data

Use Power Query Editor to shape data

Build data model

Define relationships between table. Add calculated measures and columns using DAX.

Design Visualizations

Create interactive visuals like bar charts, line graphs, and maps.

Publish and Share

Save reports locally or publish them to the Power BI Service.

Connect → Transform → Model → Visualize → Publish

Power BI Desktop streamlines the process from raw data to actionable insights, enabling better decision-making.

Power BI Desktop workflow.

From Data to Insights

Before cleaning data ...

The fundamentals of DAX, **essential functions, and practical applications in Power BI** to enhance data modeling, create custom calculations, and optimize analytics performance.

What is DAX and Why?

Definition

- DAX (Data Analysis Expressions) is a formula language used in Power BI, Power Pivot, and Analysis Services for data modeling and analysis.
- It enables the creation of calculated columns, measures, and custom tables for dynamic reporting.

Purpose of DAX:

- Perform custom calculations on your data.
- Enable dynamic aggregations that respond to slicers, filters, and visuals.
- Implement time intelligence for analyzing trends over time.
- Build powerful data models with advanced relationships and dependencies.

Key Features of DAX:

Calculated Columns:

Increases model size

➜ Created at the row level (Stored in a table, recalculates for each row).

➜ Good for precomputed values (e.g., “Profit = Sales - Cost”).

➜ Increases model size (Stored in the dataset).

Measures:

Dynamic calculation

➜ Created at the column level but calculated only when used in visuals.

➜ More efficient (reduces model size & improves performance).

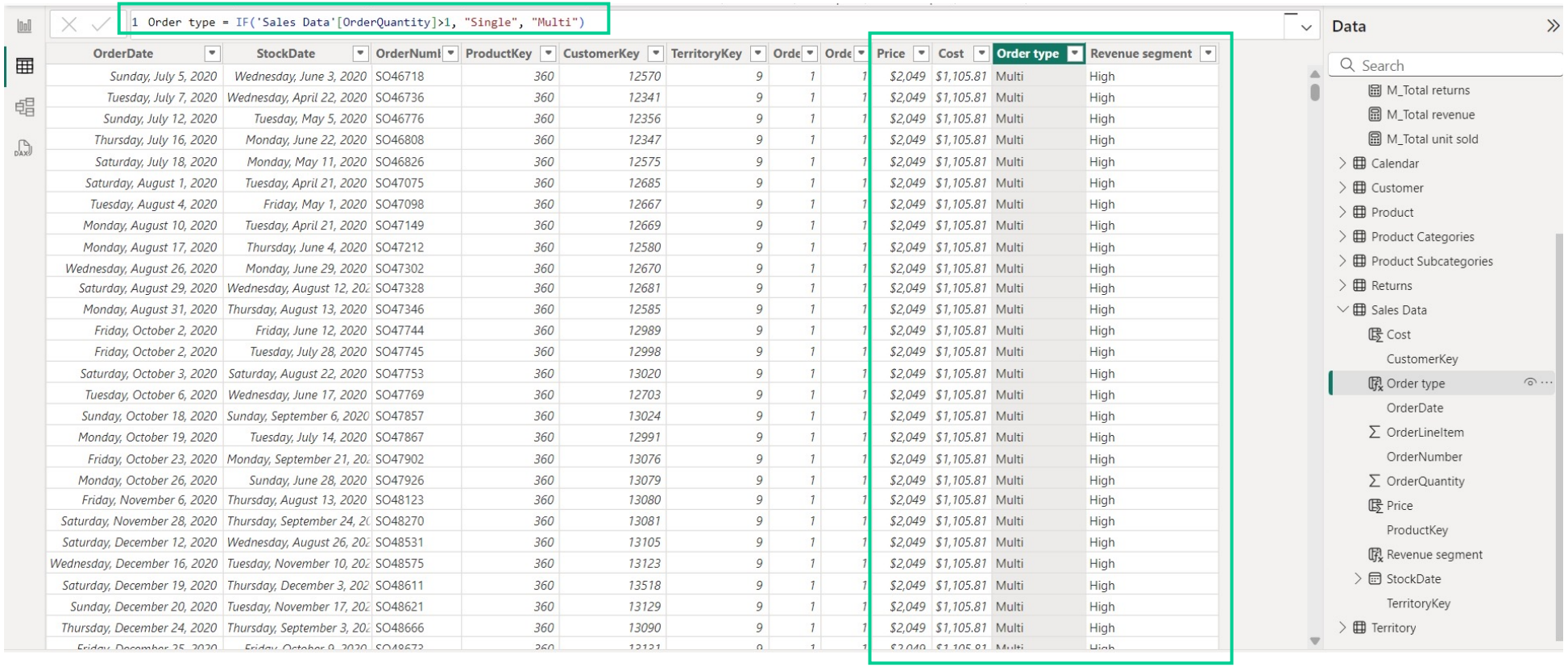
➜ Good for dynamic calculations

Calculated columns.

Enhancing your Power BI tables with row-level calculations.

Definition

- A calculated column is a new column created using DAX formulas that computes values row by row based on existing columns in the table.
- Where are they used: Adding new fields that depend on other columns (e.g., profit, full name, categories).



OrderDate	StockDate	OrderNuml	ProductKey	CustomerKey	TerritoryKey	Orde	Orde	Price	Cost	Order type	Revenue segment
Sunday, July 5, 2020	Wednesday, June 3, 2020	SO46718	360	12570	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, July 7, 2020	Wednesday, April 22, 2020	SO46736	360	12341	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, July 12, 2020	Tuesday, May 5, 2020	SO46776	360	12356	9	1	1	$2,049	$1,105.81	Multi	High
Thursday, July 16, 2020	Monday, June 22, 2020	SO46808	360	12347	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, July 18, 2020	Monday, May 11, 2020	SO46826	360	12575	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, August 1, 2020	Tuesday, April 21, 2020	SO47075	360	12685	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, August 4, 2020	Friday, May 1, 2020	SO47098	360	12667	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 10, 2020	Tuesday, April 21, 2020	SO47149	360	12669	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 17, 2020	Thursday, June 4, 2020	SO47212	360	12580	9	1	1	$2,049	$1,105.81	Multi	High
Wednesday, August 26, 2020	Monday, June 29, 2020	SO47302	360	12670	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, August 29, 2020	Wednesday, August 12, 202	SO47328	360	12681	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 31, 2020	Thursday, August 13, 2020	SO47346	360	12585	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 2, 2020	Friday, June 12, 2020	SO47744	360	12989	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 2, 2020	Tuesday, July 28, 2020	SO47745	360	12998	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, October 3, 2020	Saturday, August 22, 2020	SO47753	360	13020	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, October 6, 2020	Wednesday, June 17, 2020	SO47769	360	12703	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, October 18, 2020	Sunday, September 6, 2020	SO47857	360	13024	9	1	1	$2,049	$1,105.81	Multi	High
Monday, October 19, 2020	Tuesday, July 14, 2020	SO47867	360	12991	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 23, 2020	Monday, September 21, 20	SO47902	360	13076	9	1	1	$2,049	$1,105.81	Multi	High
Monday, October 26, 2020	Sunday, June 28, 2020	SO47926	360	13079	9	1	1	$2,049	$1,105.81	Multi	High
Friday, November 6, 2020	Thursday, August 13, 2020	SO48123	360	13080	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, November 28, 2020	Thursday, September 24, 2(	SO48270	360	13081	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, December 12, 2020	Wednesday, August 26, 202	SO48531	360	13105	9	1	1	$2,049	$1,105.81	Multi	High
Wednesday, December 16, 2020	Tuesday, November 10, 202	SO48575	360	13123	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, December 19, 2020	Thursday, December 3, 202	SO48611	360	13518	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, December 20, 2020	Tuesday, November 17, 202	SO48621	360	13129	9	1	1	$2,049	$1,105.81	Multi	High
Thursday, December 24, 2020	Thursday, September 3, 202	SO48666	360	13090	9	1	1	$2,049	$1,105.81	Multi	High

Calculated columns.

Enhancing your Power BI tables with row-level calculations.

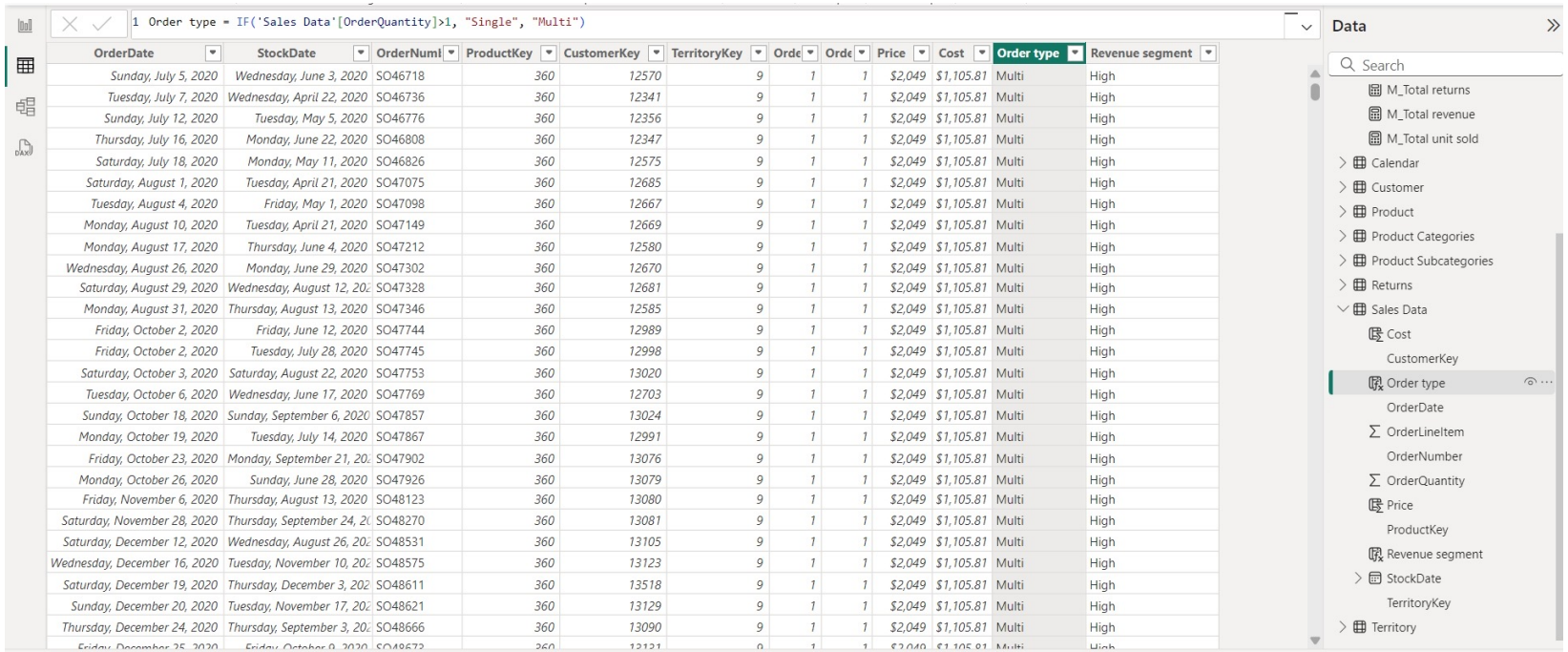
How Calculated Columns Work

Row Context:

- Each row in the table is evaluated individually when calculating the column.

Stored in the Data Model:

- Unlike measures, calculated column values are precomputed and stored.
- This can increase the file size, especially with large datasets.



OrderDate	StockDate	OrderNumb	ProductKey	CustomerKey	TerritoryKey	Orde	Orde	Price	Cost	Order type	Revenue segment
Sunday, July 5, 2020	Wednesday, June 3, 2020	SO46718	360	12570	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, July 7, 2020	Wednesday, April 22, 2020	SO46736	360	12341	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, July 12, 2020	Tuesday, May 5, 2020	SO46776	360	12356	9	1	1	$2,049	$1,105.81	Multi	High
Thursday, July 16, 2020	Monday, June 22, 2020	SO46808	360	12347	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, July 18, 2020	Monday, May 11, 2020	SO46826	360	12575	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, August 1, 2020	Tuesday, April 21, 2020	SO47075	360	12685	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, August 4, 2020	Friday, May 1, 2020	SO47098	360	12667	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 10, 2020	Tuesday, April 21, 2020	SO47149	360	12669	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 17, 2020	Thursday, June 4, 2020	SO47212	360	12580	9	1	1	$2,049	$1,105.81	Multi	High
Wednesday, August 26, 2020	Monday, June 29, 2020	SO47302	360	12670	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, August 29, 2020	Wednesday, August 12, 202	SO47328	360	12681	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 31, 2020	Thursday, August 13, 2020	SO47346	360	12585	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 2, 2020	Friday, June 12, 2020	SO47744	360	12989	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 2, 2020	Tuesday, July 28, 2020	SO47745	360	12998	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, October 3, 2020	Saturday, August 22, 2020	SO47753	360	13020	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, October 6, 2020	Wednesday, June 17, 2020	SO47769	360	12703	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, October 18, 2020	Sunday, September 6, 2020	SO47857	360	13024	9	1	1	$2,049	$1,105.81	Multi	High
Monday, October 19, 2020	Tuesday, July 14, 2020	SO47867	360	12991	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 23, 2020	Monday, September 21, 20	SO47902	360	13076	9	1	1	$2,049	$1,105.81	Multi	High
Monday, October 26, 2020	Sunday, June 28, 2020	SO47926	360	13079	9	1	1	$2,049	$1,105.81	Multi	High
Friday, November 6, 2020	Thursday, August 13, 2020	SO48123	360	13080	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, November 28, 2020	Thursday, September 24, 2	SO48270	360	13081	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, December 12, 2020	Wednesday, August 26, 202	SO48531	360	13105	9	1	1	$2,049	$1,105.81	Multi	High
Wednesday, December 16, 2020	Tuesday, November 10, 202	SO48575	360	13123	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, December 19, 2020	Thursday, December 3, 202	SO48611	360	13518	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, December 20, 2020	Tuesday, November 17, 202	SO48621	360	13129	9	1	1	$2,049	$1,105.81	Multi	High
Thursday, December 24, 2020	Thursday, September 3, 202	SO48666	360	13090	9	1	1	$2,049	$1,105.81	Multi	High

When to Use Calculated Columns?

Enhancing your Power BI tables with row-level calculations.

Static Row-Level Calculations & Classification

➜ Profit for each transaction (Revenue - Cost).

➜ Full names by combining first and last names.

```
1  order type = IF(fact_order[quantity] > 1, "Multiple", "Single")
```

customer id	payment id	purchase date	shipping id	loyalty	rating	quantity	cost	product id	status	Start of Month	order type
1047	4	Monday, December 11, 2023	3	No	3	10	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple
1265	4	Thursday, October 12, 2023	3	No	3	1	62.72919932	TABLET_SKU1002	completed	October 2023	Single
1286	4	Monday, November 20, 2023	3	No	3	4	62.72919932	TABLET_SKU1002	completed	November 2023	Multiple
1304	4	Tuesday, October 24, 2023	3	No	3	3	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple
1387	4	Monday, June 17, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	June 2024	Multiple
1478	4	Saturday, February 17, 2024	3	No	3	10	62.72919932	TABLET_SKU1002	completed	February 2024	Multiple
1603	4	Friday, May 24, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	May 2024	Multiple
1616	4	Thursday, April 11, 2024	3	No	3	5	62.72919932	TABLET_SKU1002	completed	April 2024	Multiple
1703	4	Wednesday, December 27, 2023	3	No	3	8	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple
1797	4	Sunday, October 1, 2023	3	No	3	2	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple
1817	4	Thursday, October 12, 2023	3	No	3	10	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple
2216	4	Friday, December 8, 2023	3	No	3	4	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple
2246	4	Tuesday, March 26, 2024	3	No	3	1	62.72919932	TABLET_SKU1002	completed	March 2024	Single
2290	4	Wednesday, October 4, 2023	3	No	3	7	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple
2317	4	Monday, June 24, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	June 2024	Multiple
2518	4	Friday, May 10, 2024	3	No	3	6	62.72919932	TABLET_SKU1002	completed	May 2024	Multiple
2525	4	Sunday, January 28, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	January 2024	Multiple
2659	4	Wednesday, July 10, 2024	3	No	3	7	62.72919932	TABLET_SKU1002	completed	July 2024	Multiple
2848	4	Saturday, December 16, 2023	3	No	3	9	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple
2978	4	Friday, October 6, 2023	3	No	3	10	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple
3003	4	Saturday, October 28, 2023	3	No	3	4	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple
3213	4	Saturday, February 17, 2024	3	No	3	4	62.72919932	TABLET_SKU1002	completed	February 2024	Multiple

When to Use Calculated Columns?

Enhancing your Power BI tables with row-level calculations.

Key Fields for Relationships

➜ Creating unique identifiers for joining tables.

When to Use Calculated Columns?

Enhancing your Power BI tables with row-level calculations.

Date Transformations

➜ Extracting month or year from a date field.

```
1  YEAR = YEAR(fact_order[Start of Month])
```

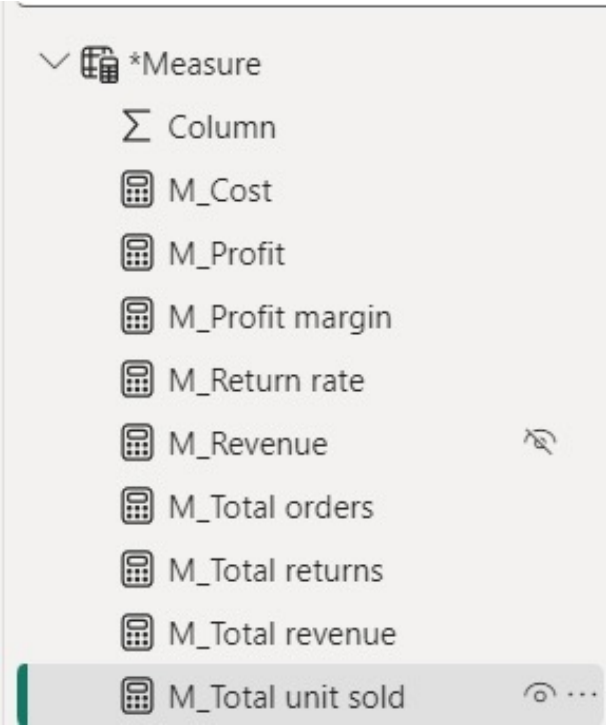
stomer id	payment id	purchase date	shipping id	loyalty	rating	quantity	cost	product id	status	Start of Month	order type	YEAR
1047	4	Monday, December 11, 2023	3	No	3	10	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple	2023
1265	4	Thursday, October 12, 2023	3	No	3	1	62.72919932	TABLET_SKU1002	completed	October 2023	Single	2023
1286	4	Monday, November 20, 2023	3	No	3	4	62.72919932	TABLET_SKU1002	completed	November 2023	Multiple	2023
1304	4	Tuesday, October 24, 2023	3	No	3	3	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple	2023
1387	4	Monday, June 17, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	June 2024	Multiple	2024
1478	4	Saturday, February 17, 2024	3	No	3	10	62.72919932	TABLET_SKU1002	completed	February 2024	Multiple	2024
1603	4	Friday, May 24, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	May 2024	Multiple	2024
1616	4	Thursday, April 11, 2024	3	No	3	5	62.72919932	TABLET_SKU1002	completed	April 2024	Multiple	2024
1703	4	Wednesday, December 27, 2023	3	No	3	8	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple	2023
1797	4	Sunday, October 1, 2023	3	No	3	2	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple	2023
1817	4	Thursday, October 12, 2023	3	No	3	10	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple	2023
2216	4	Friday, December 8, 2023	3	No	3	4	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple	2023
2246	4	Tuesday, March 26, 2024	3	No	3	1	62.72919932	TABLET_SKU1002	completed	March 2024	Single	2024
2290	4	Wednesday, October 4, 2023	3	No	3	7	62.72919932	TABLET_SKU1002	completed	October 2023	Multiple	2023
2317	4	Monday, June 24, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	June 2024	Multiple	2024
2518	4	Friday, May 10, 2024	3	No	3	6	62.72919932	TABLET_SKU1002	completed	May 2024	Multiple	2024
2525	4	Sunday, January 28, 2024	3	No	3	3	62.72919932	TABLET_SKU1002	completed	January 2024	Multiple	2024
2659	4	Wednesday, July 10, 2024	3	No	3	7	62.72919932	TABLET_SKU1002	completed	July 2024	Multiple	2024
2848	4	Saturday, December 16, 2023	3	No	3	9	62.72919932	TABLET_SKU1002	completed	December 2023	Multiple	2023

DAX Measures.

Dynamic calculations for powerful analytics.

Definition

- A measure in DAX is a formula used to perform calculations on aggregated data, dynamically adjusting based on the filter context in reports and visuals.




```
M_Total revenue = SUMX ('Sales Data', 'Sales Data'[OrderQuantity]* RELATED('Product'[ProductPrice]))

M_Total unit sold = SUM('Sales Data'[OrderQuantity])

M_Profit margin = [M_Profit]/[M_Revenue]
```

Purpose of DAX measures.

Dynamic calculations for powerful analytics.

Dynamic Calculations:

- Measures automatically update in response to applied filters and visualizations.

Efficiency:

- Measures do not contribute to the size of the data model because they are not saved.

Reusability:

- A single measure can be utilized in various visuals and reports.

Advanced Analytics:

- Allow for intricate calculations such as time intelligence and conditional aggregations.

Implicit vs. Explicit Measures.

Understanding the difference for better analytics.

Definition

- An **implicit measure** is automatically created by Power BI when you drag a numeric field into a visual's Values area.
- An explicit measure is manually created by the user using a DAX formula, offering full control over the calculation.

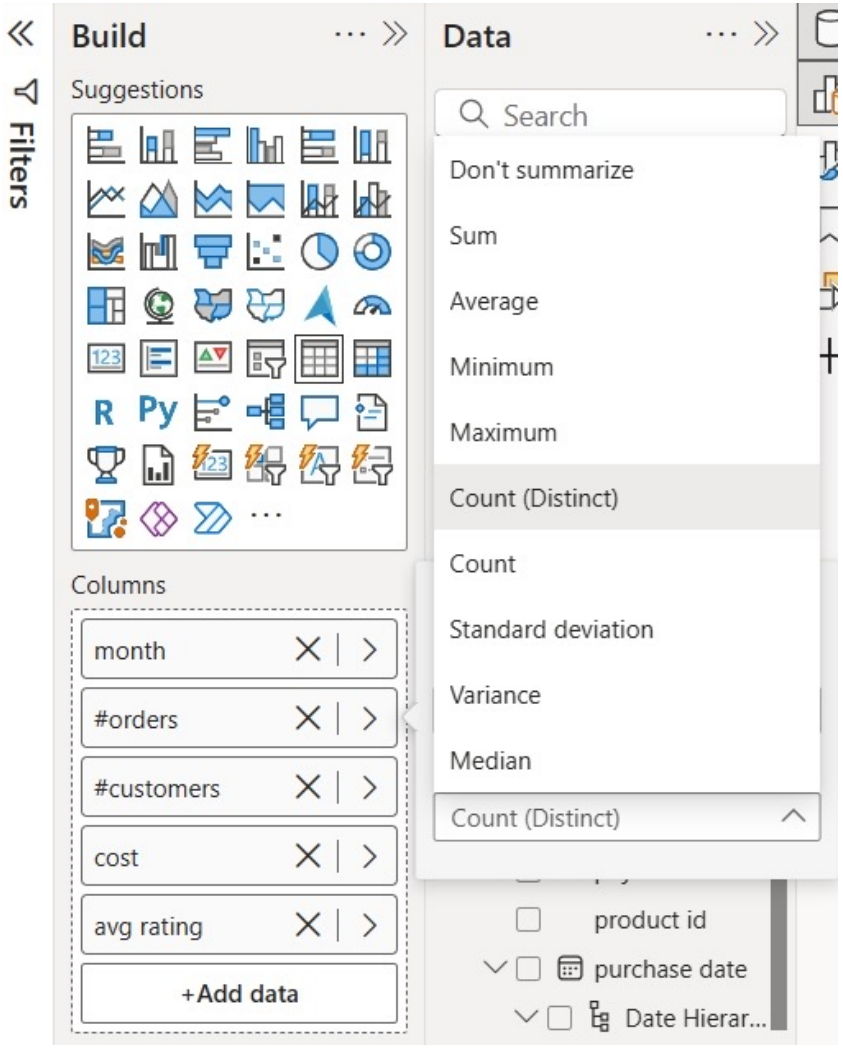
Aspect	Implicit Measures	Explicit Measures
Creation	Automatic (dragging a field into a visual).	Manual (using DAX formulas).
Customization	Limited to predefined aggregations (e.g., SUM).	Fully customizable using DAX.
Reusability	Cannot be reused outside the visual.	Can be reused across visuals and reports.
Complexity	Simple aggregations only.	Supports complex calculations.
Visibility	Exists only in the visual.	Visible in the Fields pane.
Performance	Suitable for simple needs.	Optimized for advanced scenarios.

Implicit vs. Explicit Measures.

Understanding the difference for better analytics.

Implicit measures

→ is automatically created by Power BI when you drag a numeric field into a visual's Values area.

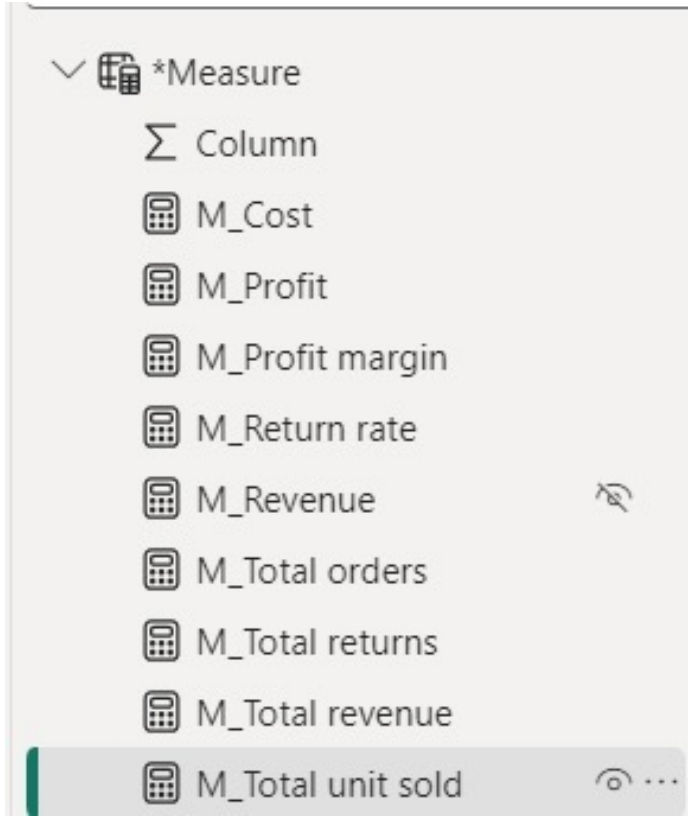


Implicit vs. Explicit Measures.

Understanding the difference for better analytics.

explicit measure

➜ is manually created by the user using a DAX formula, offering full control over the calculation.



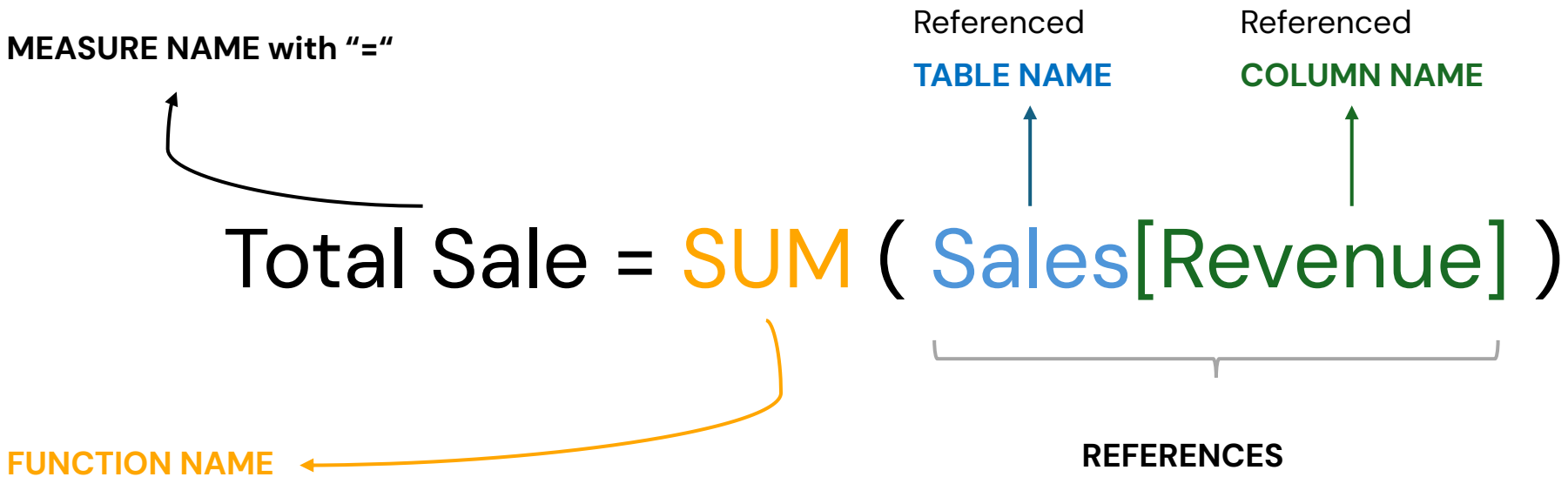
Comparison: Calculated Columns vs. Measures.

Understanding the difference for better analytics.

Aspect	Calculated Columns	Measures
Purpose	Create new fields for row-level calculations.	Perform aggregations or calculations dynamically.
Calculation Context	Works on **row context** (row by row).	Works on **filter context** (affected by slicers/filters).
Storage	Values are stored in the data model.	Values are not stored; they are computed when needed.
Reusability	Used as fields in visuals or for relationships.	Reusable across visuals and reports.
Performance Impact	Increases data model size.	More efficient as they don't store results.
Flexibility	Limited to row-level calculations.	Suitable for dynamic aggregations and complex scenarios.
Creation Context	Calculated for each row in a table.	Calculated dynamically based on the visual's filters.
Example Use Case	Calculating profit per transaction (row-level).	Calculating total sales or profit margin.

The structure of a DAX formula.

A Beginner's Guide to Writing DAX Formulas.



FUNCTION NAME

Calculated columns don't always use functions, but measures do:

- In a **Calculated Column**, **= Sales[Revenue]** returns the value from the quantity column in each row (since it evaluates one row at a time)
- In a **Measure**, **= Sales[Revenue]** will return an error since Power BI doesn't know how to translate that as a single value

REFERENCES

- Points to tables or columns in the data model.
- Format: **TableName[ColumnName].**

Overview of DAX function categories.

A Beginner's Guide to Writing DAX Formulas.

MATH & STATS Functions

*Functions used for **aggregation** or iterative, row-level calculations*

Common Examples:

- SUM
- AVERAGE
- MAX/MIN
- DIVIDE
- COUNT/COUNTA
- COUNTROWS
- DISTINCTCOUNT

Iterator Functions:

- SUMX
- AVERAGEX
- MAXX/MINX
- RANKX
- COUNTX

LOGICAL Functions

*Functions that use **conditional expressions** (IF/THEN statements)*

Common Examples:

- IF
- IFERROR
- AND
- OR
- NOT
- SWITCH
- TRUE
- FALSE

TEXT Functions

*Functions used to manipulate **text strings** or **value formats***

Common Examples:

- CONCATENATE
- COMBINEVALUES
- FORMAT
- LEFT/MID/RIGHT
- UPPER/LOWER
- LEN
- SEARCH/FIND
- REPLACE
- SUBSTITUTE
- TRIM

FILTER Functions

*Functions used to **manipulate table** and **filter contexts***

Common Examples:

- CALCULATE
- FILTER
- ALL
- ALLEXCEPT
- ALLSELECTED
- KEEPFILTERS
- REMOVEFILTERS
- SELECTEDVALUE

TABLE Functions

*Functions that **create** or **manipulate tables** and output tables vs. scalar values*

Common Examples:

- SUMMARIZE
- ADDCOLUMNS
- GENERATESERIES
- DISTINCT
- VALUES
- UNION
- INTERSECT
- TOPN

DATE & TIME Functions

*Functions used to manipulate **date & time values** or handle time intelligence calculations*

Common Examples:

- DATE
- DATEDIFF
- YEARFRAC
- YEAR/MONTH
- DAY/HOUR
- TODAY/NOW
- WEEKDAY
- WEEKNUM
- NETWORKDAYS

Time Intelligence:

- DATESYTD
- DATESMTD
- DATEADD
- DATESBETWEEN

RELATIONSHIP Functions

*Functions used to **manage & modify table relationships***

Common Examples:

- RELATED
- RELATEDTABLE
- CROSSFILTER
- USERELATIONSHIP

Note: This is NOT a comprehensive list. DAX contains more than 250 different functions!

Math Stats Functions.

A Beginner's Guide to Writing DAX Formulas.

Aggregation functions.

Perform calculations across a column or table, returning a single aggregated value.

Key functions

SUM: Adds up all values in a column.

*Example: TotalRevenue = **SUM**(Sales[Revenue])*

AVERAGE: Calculates the mean of a column.

*Example: AvgSales = **AVERAGE**(Sales[Revenue])*

MAX: Finds the highest value in a column.

*Example: MaxSales = **MAX**(Sales[Revenue])*

MIN: Finds the lowest value in a column.

*Example: MinSales = **MIN**(Sales[Revenue])*

DAX Iterators (Aggregation functions X)

Perform calculations across a column or table, returning a single aggregated value.

Key functions

Iterator Function	Description	Example Use Case
SUMX	Adds up calculated row values	`SUMX(Sales, Sales[Price] * Sales[Quantity])` (Total Revenue)
AVERAGEX	Finds the average of row-wise calculations	`AVERAGEX(Sales, Sales[Price] * Sales[Quantity])` (Avg. Order Value)
MAXX	Finds the max value after applying row-wise calculation	`MAXX(Sales, Sales[Profit])` (Max Profit)
MINX	Finds the min value after applying row-wise calculation	`MINX(Sales, Sales[Cost])` (Min Cost)
COUNTX	Counts the number of rows meeting a condition	`COUNTX(Sales, Sales[Quantity] > 2)` (Orders with more than 2 items)
FILTER	Filters rows based on a condition	`FILTER(Sales, Sales[Profit] > 100)` (High-Profit Orders)

DAX Iterators (Aggregation functions X)

Perform calculations across a column or table, returning a single aggregated value.

Example: Why Use an Iterator?

Let's say we have a Sales Table with Price and Quantity columns:

Question: How do we calculate Total Sales Revenue (Price * Quantity)?

Product	Price ($)	Quantity
Laptop	1,000	2
Mouse	30	5
Keyboard	50	3

❌ Using Standard Aggregation (Incorrect Approach)

```
Total Sales = SUM(Sales[Price]) * SUM(Sales[Quantity])
```

- Problem: This formula multiplies column totals instead of row-level values.
- Incorrect Calculation:
  - SUM(Price) = 1,000 + 30 + 50 = 1,080
  - SUM(Quantity) = 2 + 5 + 3 = 10
  - 1,080 * 10 = 10,800 (Incorrect!)

DAX Iterators (Aggregation functions X)

Perform calculations across a column or table, returning a single aggregated value.

Example: Why Use an Iterator?

Let's say we have a Sales Table with Price and Quantity columns:

Question: How do we calculate Total Sales Revenue (Price * Quantity)?

Product	Price ($)	Quantity
Laptop	1,000	2
Mouse	30	5
Keyboard	50	3

✅ **Using an Iterator Function (Correct Approach)**

Total Sales = **SUMX**(Sales, Sales[Price] * Sales[Quantity])

- SUMX iterates row by row:
  - Laptop: 1,000 × 2 = 2,000
  - Mouse: 30 × 5 = 150
  - Keyboard: 50 × 3 = 150
- Correct Total: 2,000 + 150 + 150 = 2,300

Filtered Iterators: Using SUMX with FILTER

Perform calculations across a column or table, returning a single aggregated value.

📌 **Scenario:**

A retail company wants to sum the sales only for high-value products (Price > 500).

❌ **Incorrect Approach (SUM on the Whole Table)**

```
High Value Sales = SUM(Sales[Revenue])
```

✅ **Correct Approach Using SUMX + FILTER**

```
High Value Sales = SUMX(FILTER(Sales, Sales[Price] > 500), Sales[Price] * Sales[Quantity])
```

Explanation:

- FILTER(Sales, Sales[Price] > 500): Selects only rows where Price > 500.
- SUMX(..., Sales[Price] * Sales[Quantity]): Calculates revenue only for those rows.

Counting functions.

Count rows or specific values in a column.

Key functions

COUNT: Counts non-blank rows in a column.

Example: *TotalTransactions = **COUNT**(Sales[TransactionID])*

COUNTA: Counts non-blank rows, including text and logical values.

Example: *TotalCustomers = **COUNTA**(Customers[CustomerName])*

COUNTBLANK: Counts blank values in a column.

Example: *MissingData = **COUNTBLANK**(Sales[Revenue])*

DISTINCTCOUNT: Counts unique values in a column.

Example: *UniqueCustomers = **DISTINCTCOUNT**(Sales[CustomerID])*

Statistical functions.

Perform statistical analysis on data columns..

Key functions

MEDIAN: Returns the median value in a column.

Example: *MedianSales = **MEDIAN**(Sales[Revenue])*

STDEV.P: Calculates the standard deviation of an entire population.

Example: *RevenueStDev = **STDEV.P**(Sales[Revenue])*

STDEV.S: Calculates the standard deviation of a sample.

Example: *SampleStDev = **STDEV.S**(Sales[Revenue])*

PRODUCT: Multiplies all values in a column.

Example: *TotalProduct = **PRODUCT**(Sales[GrowthFactor])*

Logical functions.

Logical functions in DAX are used to evaluate conditions and return values based on the results of those evaluations.

Key functions

IF: Performs conditional logic and returns one value if a condition is true and another if false..

Example: *HighSales = **IF**(Sales[Revenue] > 5000, "High", "Low")*

IFERROR: Returns a value if an expression results in an error.

Example: *SafeDivision = **IFERROR**(Sales[Revenue] / Sales[Quantity], 0)*

AND / OR: Combine multiple conditions.

Example: *HighProfit = **IF**(**AND**(Sales[Revenue] > 5000, Sales[Profit] > 2000), "Yes", "No")*

SWITCH: Evaluates an expression against multiple conditions and returns a value.

Example: *SalesCategory = **SWITCH**(TRUE(), Sales[Revenue] > 10000, "Platinum",*
Sales[Revenue] > 5000, "Gold", "Silver")

What is the SWITCH Function?

The SWITCH function evaluates an expression and returns a result based on matching predefined values.

It is used to simplify multiple IF statements and improve readability & performance.

```
SWITCH(Expression,
        Value1, Result1,
        Value2, Result2,
        Value3, Result3,
        ...,
        DefaultResult)
```

2️⃣ Example: Categorizing Sales Performance

❌ Using Nested IF Statements (Difficult to Read)

DAX Copy Edit

```
Sales Category =
IF(Sales[Total Revenue] > 50000, "High",
   IF(Sales[Total Revenue] > 20000, "Medium",
      "Low"))
```

- **Problem:** Hard to read & maintain when conditions increase.

✅ Using SWITCH (Cleaner & More Efficient)

DAX Copy Edit

```
Sales Category =
SWITCH(TRUE(),
    Sales[Total Revenue] > 50000, "High",
    Sales[Total Revenue] > 20000, "Medium",
    "Low"
)
```

What is the SWITCH Function?

3️⃣ Example: Mapping Product Categories

📌 **Scenario:** A company wants to classify products into **categories** based on their Product ID.

Using SWITCH to Assign Product Categories

DAX Copy Edit

```
Product Category =
SWITCH(Sales[Product ID],
    101, "Laptops",
    102, "Monitors",
    103, "Keyboards",
    104, "Mice",
    "Other"
)
```

📌 **Explanation:**

✅ If `Product ID = 101`, it returns `"Laptops"`.

✅ If `Product ID = 102`, it returns `"Monitors"`.

✅ If no match is found, it returns `"Other"`.

Why Use IF/SWITCH in DAX When We Can Use Conditional Columns in Power Query?

Power Query vs. DAX: The Key Differences

Feature	Power Query (M Language)	DAX (Report View)
Where it runs	Runs at the **data transformation level** (before loading to the model).	Runs at the **visualization level** (after data is loaded).
When it updates	Applied **once during data refresh**.	Updates **dynamically based on user interaction** (slicers, filters).
Performance Impact	Reduces model size (pre-computed before loading).	Calculated on-the-fly, which can slow performance on large datasets.
Context Awareness	Cannot respond to slicers or dynamic filters.	Can dynamically adjust values based on filters & visuals.
Best Used For	**Static classifications** (e.g., "New Customer" or "High Revenue Product").	**Dynamic calculations** (e.g., "Total Sales for Selected Month").

Power Query vs. DAX: The Key Differences

2️⃣ When to Use Power Query for IF-ELSE Logic?

✅ **For static calculations that don't change with filters.**

✅ **For pre-processed classifications (e.g., category assignments).**

✅ **To reduce memory usage & improve performance.**

📌 **Example (Power Query Conditional Column):**

- If we want to classify customers based on **total purchases**, we can add a **calculated column** in Power Query:

Power

Copy Edit

```
if [Total Purchases] > 5000 then "VIP"
else if [Total Purchases] > 1000 then "Regular"
else "New Customer"
```

- **Best for:**
  - Customer segmentation
  - Product categories
  - Fixed discount tiers

Power Query vs. DAX: The Key Differences

3️⃣ When to Use DAX IF/SWITCH in Report View?

✅ **For calculations that depend on user selections (filters, slicers, or visuals).**

✅ **When you need dynamic calculations that change in real time.**

✅ **When relationships between tables require on-the-fly adjustments.**

📌 **Example (DAX in Report View - Dynamic Calculation):**

- Suppose we want to **classify customers dynamically based on their purchases in the selected period** (e.g., the last 6 months):

DAX Copy Edit

```
Customer Status =
SWITCH(
    TRUE(),
    SUM(Sales[Total Purchases]) > 5000, "VIP",
    SUM(Sales[Total Purchases]) > 1000, "Regular",
    "New Customer"
)
```

- **Best for:**
  - Dynamic ranking of customers based on **selected timeframe**
  - Conditional formatting in Power BI based on filter selection

Text Functions.

A Beginner's Guide to Writing DAX Formulas.

Text functions.

Powerful Tools for Text Manipulation.

Key functions

CONCATENATE: Combines two text strings into one.

Example: *FullName = **CONCATENATE**(Employees[FirstName], " " & Employees[LastName])*

COMBINEVALUES: Combines multiple text strings with a delimiter..

Example: *FullName = **COMBINEVALUES**(" ", Employees[FirstName], Employees[LastName])*

FORMAT: Converts a value to a specific format.

Example: *FormattedDate = **FORMAT**(Orders[OrderDate], "DD-MM-YYYY")*

LEFT, MID, RIGHT: Extract parts of a string.

Example: *First3Chars = **LEFT**(Products[ProductCode], 3)*

*MiddlePart = **MID**(Products[ProductCode], 4, 5)*

*Last2Chars = **RIGHT**(Products[ProductCode], 2).*

Text functions.

Powerful Tools for Text Manipulation.

Key functions

SEARCH/FIND: Locate a substring within a text string.

Example: *Position = **SEARCH**("-", Products[SKU], 1)*

REPLACE: Replace part of a string with new text based on position.

Example: *ReplacePart = **REPLACE**("ABC123", 1, 3, "XYZ")*

SUBSTITUTE: Replace occurrences of specific text.

Example: *ReplaceDash = **SUBSTITUTE**("123-456-789", "-", "/")*

TRIM: Removes extra spaces from text.

Example: *CleanedText = **TRIM**(Customers[Name]).*

Date & Time Functions.

A Beginner's Guide to Writing DAX Formulas.

Date & Time Functions in DAX.

Powerful Tools for Time-Based Analysis.

Date & Time functions in DAX allow you to manipulate, analyze, and perform calculations on date and time data.

YEAR / MONTH / DAY: Extract components of a date..

Example: *Year = YEAR(Orders[OrderDate])*

WEEKDAY: Returns the day of the week as a number.

Example: *DayOfWeek = WEEKDAY(Orders[OrderDate], 2) -- 2: Monday as the first day*

FORMAT: Converts a date to a formatted string.

Example: *FormattedDate = FORMAT(Orders[OrderDate], "DD-MM-YYYY")*

TODAY: Returns the current date.

Example: *DaysSinceOrder = TODAY() - Orders[OrderDate]*

Date & Time Functions in DAX.

Powerful Tools for Time-Based Analysis.

Date & Time functions in DAX allow you to manipulate, analyze, and perform calculations on date and time data.

EDATE / EOMONTH: Adds/subtracts months to/from a date.

Example: *NextMonth = EDATE(Orders[OrderDate], 1)*

MonthEnd = EOMONTH(Orders[OrderDate], 0)

Time Intelligence Functions.

Powerful Tools for Time-Based Analysis.

Date & Time functions in DAX allow you to manipulate, analyze, and perform calculations on date and time data.

DATEDIFF: Returns the difference between two dates.

Example: *DaysBetween = DATEDIFF(Orders[OrderDate], TODAY(), DAY)*

CALENDAR / CALENDARAUTO: Creates a table of dates.

Example: *CalendarTable = CALENDAR(DATE(2022, 1, 1), DATE(2022, 12, 31))*

Most Common Practices.

A Beginner's Guide to Writing DAX Formulas.

Practical Use Cases.

Powerful Tools for Time-Based Analysis.

Aggregation Over Specific Periods

TOTALYTD: Calculates the year-to-date total for a measure.

Syntax: TOTALYTD(<Expression>, <Dates>, [<Filter>], [<YearEndDate>])

*Example: SalesYTD = **TOTALYTD**(SUM(Sales[Revenue]), Dates[Date])*

TOTALQTD: Calculates the quarter-to-date total for a measure.

Syntax: TOTALQTD (<Expression>, <Dates>, [<Filter>], [<YearEndDate>])

*Example: SalesQTD = **TOTALQTD** (SUM(Sales[Revenue]), Dates[Date])*

TOTALMTD: Calculates the month-to-date total for a measure.

Syntax: TOTALMTD (<Expression>, <Dates>, [<Filter>], [<YearEndDate>])

*Example: SalesYTD = **TOTALMTD** (SUM(Sales[Revenue]), Dates[Date])*

Practical Use Cases.

Powerful Tools for Time-Based Analysis.

Previous Period Calculations

PREVIOUSYEAR: Returns a table with the same period in the previous year.

Syntax: PREVIOUSYEAR(<Dates>, [<YearEndDate>])

Example: SalesPreviousYear = CALCULATE(SUM(Sales[Revenue]), PREVIOUSYEAR(Dates[Date]))

PREVIOUSQUARTER: Returns the same period in the previous quarter.

Syntax: PREVIOUSQUARTER(<Dates>)

PREVIOUSMONTH: Returns the same period in the previous month.

Syntax: PREVIOUSMONTH(<Dates>)

PREVIOUSDAY: Returns the same period in the previous day.

Syntax: PREVIOUSDAY(<Dates>)

Practical Use Cases.

Powerful Tools for Time-Based Analysis.

Next Period Calculations

NEXTYEAR: Returns the same period in the next year.

Syntax: NEXTYEAR(<Dates>)

NEXTQUARTER: Returns the same period in the next quarter.

Syntax: NEXTQUARTER (<Dates>)

NEXTMONTH: Returns the same period in the next month.

Syntax: NEXTMONTH (<Dates>)

Practical Use Cases.

Powerful Tools for Time-Based Analysis.

Rolling and Moving Averages

These functions calculate running totals or moving averages.

DATESYTD (<Dates>, [<YearEndDate>]): Returns the dates year-to-date, quarter-to-date, or month-to-date.

*Syntax: RollingYTD = CALCULATE(SUM(Sales[Revenue]), **DATESYTD**(Dates[Date]))*

FIRSTDATE / LASTDATE: Returns the first or last date in a table..

Syntax: FIRSTDATE(<Dates>)

LASTDATE(<Dates>)

Relationship Functions.

A Beginner's Guide to Writing DAX Formulas.

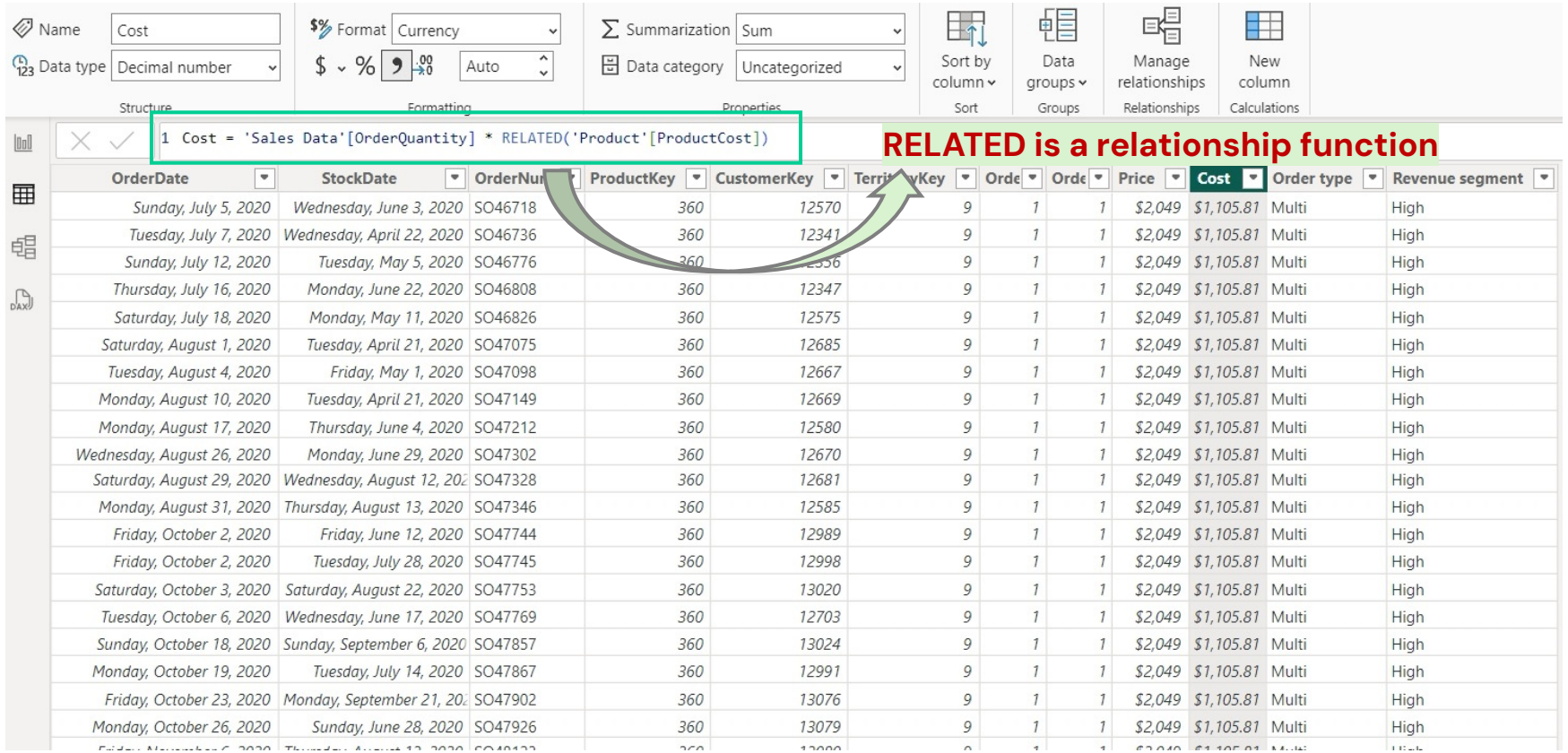
Relationship Function.

Managing Relationships for Data Analysis in Power BI.

What Are Relationship Functions?

Relationship functions in DAX allow you to work with relationships between tables in your data model.

They help retrieve related values, manage filtering, and adjust relationship behaviors dynamically.



OrderDate	StockDate	OrderNu	ProductKey	CustomerKey	Terri yKey	Orde	Orde	Price	Cost	Order type	Revenue segment
Sunday, July 5, 2020	Wednesday, June 3, 2020	SO46718	360	12570	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, July 7, 2020	Wednesday, April 22, 2020	SO46736	360	12341	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, July 12, 2020	Tuesday, May 5, 2020	SO46776	360	2356	9	1	1	$2,049	$1,105.81	Multi	High
Thursday, July 16, 2020	Monday, June 22, 2020	SO46808	360	12347	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, July 18, 2020	Monday, May 11, 2020	SO46826	360	12575	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, August 1, 2020	Tuesday, April 21, 2020	SO47075	360	12685	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, August 4, 2020	Friday, May 1, 2020	SO47098	360	12667	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 10, 2020	Tuesday, April 21, 2020	SO47149	360	12669	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 17, 2020	Thursday, June 4, 2020	SO47212	360	12580	9	1	1	$2,049	$1,105.81	Multi	High
Wednesday, August 26, 2020	Monday, June 29, 2020	SO47302	360	12670	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, August 29, 2020	Wednesday, August 12, 20	SO47328	360	12681	9	1	1	$2,049	$1,105.81	Multi	High
Monday, August 31, 2020	Thursday, August 13, 2020	SO47346	360	12585	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 2, 2020	Friday, June 12, 2020	SO47744	360	12989	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 2, 2020	Tuesday, July 28, 2020	SO47745	360	12998	9	1	1	$2,049	$1,105.81	Multi	High
Saturday, October 3, 2020	Saturday, August 22, 2020	SO47753	360	13020	9	1	1	$2,049	$1,105.81	Multi	High
Tuesday, October 6, 2020	Wednesday, June 17, 2020	SO47769	360	12703	9	1	1	$2,049	$1,105.81	Multi	High
Sunday, October 18, 2020	Sunday, September 6, 2020	SO47857	360	13024	9	1	1	$2,049	$1,105.81	Multi	High
Monday, October 19, 2020	Tuesday, July 14, 2020	SO47867	360	12991	9	1	1	$2,049	$1,105.81	Multi	High
Friday, October 23, 2020	Monday, September 21, 20	SO47902	360	13076	9	1	1	$2,049	$1,105.81	Multi	High
Monday, October 26, 2020	Sunday, June 28, 2020	SO47926	360	13079	9	1	1	$2,049	$1,105.81	Multi	High

Relationship Function.

Managing Relationships for Data Analysis in Power BI.

Key functions:

RELATED: Returns a single value from a related table for the current row.

- *ProductName = **RELATED**(Products[ProductName])*

RELATEDTABLE: Returns a table of all rows related to the current row.

- *TotalSalesPerProduct = SUMX(RELATEDTABLE(Sales), Sales[Revenue])*

USERELATIONSHIP: Forces DAX to use an inactive relationship instead of the active one.

- *SalesByDeliveryDate = CALCULATE(SUM(Sales[Revenue]), **USERELATIONSHIP**(Sales[DeliveryDate], Dates[Date]))*

CROSSFILTER: Modifies the cross-filter direction of a relationship dynamically.

- *AdjustedSales = CALCULATE(SUM(Sales[Revenue]), **CROSSFILTER**(Products[ProductID], Sales[ProductID], Both))*

Create Calendar table.

.

Use M code in Power query editor: Tạo bảng trống → Advanced editor → Nhập M code

```
let
    // Lấy ngày bắt đầu và ngày kết thúc từ bảng Fact Table
    StartDate = List.Min(FactTable(Order_Date)),
    EndDate = List.Max(FactTable(Order_Date)),
    // Tạo danh sách ngày từ StartDate đến EndDate
    DateList = List.Dates(StartDate, Duration.Days(EndDate - StartDate) + 1, #duration(1, 0, 0, 0)),
    // Chuyển danh sách ngày thành bảng
    DateTable = Table.FromList(DateList, Splitter.SplitByNothing(), {"Date"}),


    // Chuyển đổi cột Date sang kiểu dữ liệu Date
    DateTableWithColumns = Table.TransformColumnTypes(DateTable, {{"Date", type date}})
in
    DateTableWithColumns
```

Create Calendar table.

.

Tạo bảng calendar trên Front-end: Modeling > New Table.

```
CalendarTable =
ADDCOLUMNS(
    CALENDAR(
        MIN(FactTable[Order_Date]),
        MAX(FactTable[Order_Date])
    ),
    "Year", YEAR([Date]),
    "Month", FORMAT([Date], "MMMM"),
    "MonthNumber", MONTH([Date]),
    "Quarter", "Q" & FORMAT([Date], "Q"),
    "Weekday", FORMAT([Date], "dddd"),
    "WeekNumber", WEEKNUM([Date])
)
```

Filter Functions.

A Beginner's Guide to Writing DAX Formulas.

CALCULATE, FILTER, ALL, ALLEXCEPT, ALLSELECTED, ...

Filter Functions.

A Beginner's Guide to Writing DAX Formulas.

CALCULATE, FILTER, ALL, ALLEXCEPT, ALLSELECTED, ...

DAX FILTER functions allow us to manipulate tables and control filter contexts in Power BI calculations. These functions help refine data before applying aggregations, giving more control over what data is included or excluded in calculations.

CALCULATE Function.

The Most Powerful DAX Function

What is CALCULATE?

The CALCULATE function evaluates an expression in a modified filter context, allowing you to change, add, or remove filters dynamically.

Syntax: CALCULATE(<*Expression*>, <*Filter1*>, <*Filter2*>, ...)

Expression: The aggregation or calculation you want to perform.

Filter: One or more conditions that modify the filter context.

Example

Basic Example: Applying a Filter:

- SalesEast = CALCULATE(SUM(Sales[Revenue]), **Sales[Region] = "East"**)

Using Multiple Filters:

- HighValueSalesEast = CALCULATE(SUM(Sales[Revenue]), **Sales[Region] = "East", Sales[Revenue] > 500**)

Using FILTER for Complex Logic:

- SalesAboveCost = CALCULATE(SUM(Sales[Revenue]), FILTER**(Sales, Sales[Cost] >** RELATED**(Product[Target_Cost]))**)

FILTER Function.

Row-by-Row Table Filtering.

What is FILTER?

The FILTER function returns a table containing rows that meet a specified condition.

It is evaluated row by row and often used inside functions like CALCULATE, SUMX, or COUNTROWS.

Syntax: FILTER(<*Table*>, <*Condition*>)

Table: The table to be filtered.

Condition: A logical expression that determines which rows to keep.

Example

Using FILTER Inside CALCULATE::

- SalesAbove1000 = CALCULATE(SUM(Sales[Revenue]), FILTER(Sales, Sales[Revenue] > 1000))

Using Multiple Filters:

- HighValueSalesEast = CALCULATE(SUM(Sales[Revenue]), **Sales[Region] = "East", Sales[Revenue] > 500**)

FILTER with ALL for Custom Totals:

- SalesIgnoreProduct = CALCULATE(SUM(Sales[Revenue]), FILTER(ALL(Sales[Product]), Sales[Region] = "East"))

ALL Function.

Removes All Filters on a Table or Column.

What is ALL?

- Ignores existing filters and returns all rows in a table or column.
- Used when we need global calculations (e.g., grand total comparisons)..

Syntax: ALL*(<Table or Column>)*

Table: The table to be not filtered.

Example

Using ALL Inside CALCULATE:

- *Total Sales (No Filters)* = CALCULATE(SUM(Sales[Amount]), **ALL**(Sales))
- *% of Total Sales* = DIVIDE(SUM(Sales[Amount]), CALCULATE(SUM(Sales[Amount]), **ALL**(Sales)))
    - *ALL(Sales) removes any active filters (e.g., product, region, date).*
    - *The measure always returns total sales across all filters.*
    - *Useful for % of Total Sales per category.*

ALLEXCEPT Function.

Removes All Filters Except Specific Columns.

What is ALLEXCEPT?

- Removes all filters except the ones specified.
- Useful when keeping a specific filter while ignoring others.

Syntax: ALLEXCEPT(*<Table, Column1, Column2, …>*)

Table: columns will be not filtered.

Example:

Total Sales by Year (Ignoring Region):

- *Total Sales by Year = CALCULATE(SUM(Sales[Amount]), **ALLEXCEPT**(Sales, Sales[Year]))*
  - *All filters (e.g., Region, Product) are removed.*
  - *Only Year remains as a filter.*
  - *Useful for yearly comparisons while ignoring other filters.*

REMOVEFILTERS Function.

Clears Filters from a Column/Table While Keeping Others.

What is REMOVEFILTERS?

- Similar to ALL(), but more selective.
- Clears filters only from specific columns.

Syntax: REMOVEFILTERS*(<Table, Column1, Column2, …>)*

Table: columns will be not filtered.

Example:

Sales Ignoring Product Filters:

- *Total Sales (Ignore Product) = CALCULATE(SUM(Sales[Amount]), **REMOVEFILTERS**(Sales[Product]))*
  - *If the user filters by Product, this measure ignores that filter.*
  - *Region, Date, and other filters still apply.*
  - *Used in multi-filter reports where certain filters need to be ignored.*

SELECTEDVALUE Function.

Returns the User's Selection from a Slicer.

What is SELECTEDVALUE?

- Returns the selected value from a column (or a default value if multiple are selected).
- Used in dynamic report titles & calculations.

Syntax: REMOVEFILTERS(*<Table, Column1, Column2, …>*)

Table: columns will be not filtered.

Example:

Dynamic Title Based on Selected Year:

- *Report Title = "Sales Report for " & SELECTEDVALUE(Sales[Year], "All Years"*
    - *If 2023 is selected in a slicer, the title updates to "Sales Report for 2023".*
    - *If no selection is made, it defaults to "Sales Report for All Years".*

Practicing.

A Beginner's Guide to Writing DAX Formulas.

Time Intelligence Practice

Year	Month	revenue	M_revenue_YTD	M_revenue_YTD_2	M_revenue_YTD_3	M_total_revenue_year	M_%_revenue_in_total_year	M_revenue_lastmonth	M_revenue_previous_month	M_revenue_MOM
2023	9	0.48M	481,961.79	481,961.79	481,961.79	6,849,562.61	7.04%			0.0%
2023	10	2.32M	2,800,428.14	2,800,428.14	2,800,428.14	6,849,562.61	33.85%	481,961.79	481,961.79	381.0%
2023	11	2.07M	4,868,862.28	4,868,862.28	4,868,862.28	6,849,562.61	30.20%	2,318,466.35	2,318,466.35	-10.8%
2023	12	1.98M	6,849,562.61	6,849,562.61	6,849,562.61	6,849,562.61	28.92%	2,068,434.14	2,068,434.14	-4.2%
2024	1	6.62M	6,620,343.20	6,620,343.20	13,469,905.81	56,753,276.49	11.67%	1,980,700.33	1,980,700.33	234.2%
2024	2	5.73M	12,354,039.26	12,354,039.26	19,203,601.87	56,753,276.49	10.10%	6,620,343.20	6,620,343.20	-13.4%
2024	3	6.32M	18,678,407.10	18,678,407.10	25,527,969.71	56,753,276.49	11.14%	5,733,696.06	5,733,696.06	10.3%
2024	4	6.42M	25,096,660.72	25,096,660.72	31,946,223.33	56,753,276.49	11.31%	6,324,367.84	6,324,367.84	1.5%
2024	5	6.71M	31,805,703.65	31,805,703.65	38,655,266.26	56,753,276.49	11.82%	6,418,253.62	6,418,253.62	4.5%
2024	6	6.67M	38,474,337.24	38,474,337.24	45,323,899.85	56,753,276.49	11.75%	6,709,042.93	6,709,042.93	-0.6%
2024	7	6.54M	45,009,466.76	45,009,466.76	51,859,029.37	56,753,276.49	11.51%	6,668,633.59	6,668,633.59	-2.0%
2024	8	6.71M	51,715,585.37	51,715,585.37	58,565,147.98	56,753,276.49	11.82%	6,535,129.52	6,535,129.52	2.6%
2024	9	5.04M	56,753,276.49	56,753,276.49	63,602,839.10	56,753,276.49	8.88%	6,706,118.61	6,706,118.61	-24.9%
Total		**63.60M**	**56,753,276.49**	**56,753,276.49**	**63,602,839.10**	**63,602,839.10**	**100.00%**		**58,565,147.98**	**0.0%**

Time Intelligence Practice.

How to make some advanced metrics?

Year	Month	revenue	M_revenue_YTD	M_revenue_YTD_2	M_revenue_YTD_3	M_total_revenue_year	M_%_revenue_in_total_year	M_revenue_lastmonth	M_revenue_previous_month	M_revenue_MOM
2023	9	0.48M	481,961.79	481,961.79	481,961.79	6,849,562.61	7.04%			0.0%
2023	10	2.32M	2,800,428.14	2,800,428.14	2,800,428.14	6,849,562.61	33.85%	481,961.79	481,961.79	381.0%
2023	11	2.07M	4,868,862.28	4,868,862.28	4,868,862.28	6,849,562.61	30.20%	2,318,466.35	2,318,466.35	-10.8%
2023	12	1.98M	6,849,562.61	6,849,562.61	6,849,562.61	6,849,562.61	28.92%	2,068,434.14	2,068,434.14	-4.2%
2024	1	6.62M	6,620,343.20	6,620,343.20	13,469,905.81	56,753,276.49	11.67%	1,980,700.33	1,980,700.33	234.2%
2024	2	5.73M	12,354,039.26	12,354,039.26	19,203,601.87	56,753,276.49	10.10%	6,620,343.20	6,620,343.20	-13.4%
2024	3	6.32M	18,678,407.10	18,678,407.10	25,527,969.71	56,753,276.49	11.14%	5,733,696.06	5,733,696.06	10.3%
2024	4	6.42M	25,096,660.72	25,096,660.72	31,946,223.33	56,753,276.49	11.31%	6,324,367.84	6,324,367.84	1.5%
2024	5	6.71M	31,805,703.65	31,805,703.65	38,655,266.26	56,753,276.49	11.82%	6,418,253.62	6,418,253.62	4.5%
2024	6	6.67M	38,474,337.24	38,474,337.24	45,323,899.85	56,753,276.49	11.75%	6,709,042.93	6,709,042.93	-0.6%
2024	7	6.54M	45,009,466.76	45,009,466.76	51,859,029.37	56,753,276.49	11.51%	6,668,633.59	6,668,633.59	-2.0%
2024	8	6.71M	51,715,585.37	51,715,585.37	58,565,147.98	56,753,276.49	11.82%	6,535,129.52	6,535,129.52	2.6%
2024	9	5.04M	56,753,276.49	56,753,276.49	63,602,839.10	56,753,276.49	8.88%	6,706,118.61	6,706,118.61	-24.9%
Total		**63.60M**	**56,753,276.49**	**56,753,276.49**	**63,602,839.10**	**63,602,839.10**	**100.00%**		**58,565,147.98**	**0.0%**

```
M_total_revenue_year =
    CALCULATE(
        [M_Revenue_order],
        ALLEXCEPT(dim_calendar, dim_calendar[Year])
        )
```

Time Intelligence Practice.

How to make some advanced metrics?

Year	Month	revenue	M_revenue_YTD	M_revenue_YTD_2	M_revenue_YTD_3	M_total_revenue_year	M_%_revenue_in_total_year	M_revenue_lastmonth	M_revenue_previous_month	M_revenue_MOM
2023	9	0.48M	481,961.79	481,961.79	481,961.79	6,849,562.61	7.04%			0.0%
2023	10	2.32M	2,800,428.14	2,800,428.14	2,800,428.14	6,849,562.61	33.85%	481,961.79	481,961.79	381.0%
2023	11	2.07M	4,868,862.28	4,868,862.28	4,868,862.28	6,849,562.61	30.20%	2,318,466.35	2,318,466.35	-10.8%
2023	12	1.98M	6,849,562.61	6,849,562.61	6,849,562.61	6,849,562.61	28.92%	2,068,434.14	2,068,434.14	-4.2%
2024	1	6.62M	6,620,343.20	6,620,343.20	13,469,905.81	56,753,276.49	11.67%	1,980,700.33	1,980,700.33	234.2%
2024	2	5.73M	12,354,039.26	12,354,039.26	19,203,601.87	56,753,276.49	10.10%	6,620,343.20	6,620,343.20	-13.4%
2024	3	6.32M	18,678,407.10	18,678,407.10	25,527,969.71	56,753,276.49	11.14%	5,733,696.06	5,733,696.06	10.3%
2024	4	6.42M	25,096,660.72	25,096,660.72	31,946,223.33	56,753,276.49	11.31%	6,324,367.84	6,324,367.84	1.5%
2024	5	6.71M	31,805,703.65	31,805,703.65	38,655,266.26	56,753,276.49	11.82%	6,418,253.62	6,418,253.62	4.5%
2024	6	6.67M	38,474,337.24	38,474,337.24	45,323,899.85	56,753,276.49	11.75%	6,709,042.93	6,709,042.93	-0.6%
2024	7	6.54M	45,009,466.76	45,009,466.76	51,859,029.37	56,753,276.49	11.51%	6,668,633.59	6,668,633.59	-2.0%
2024	8	6.71M	51,715,585.37	51,715,585.37	58,565,147.98	56,753,276.49	11.82%	6,535,129.52	6,535,129.52	2.6%
2024	9	5.04M	56,753,276.49	56,753,276.49	63,602,839.10	56,753,276.49	8.88%	6,706,118.61	6,706,118.61	-24.9%
Total		**63.60M**	**56,753,276.49**	**56,753,276.49**	**63,602,839.10**	**63,602,839.10**	**100.00%**		**58,565,147.98**	**0.0%**

```
M_revenue_lastmonth =
    CALCULATE(
        [M_Revenue_order],
        PREVIOUSMONTH(dim_calendar[Date])
        )
```

```
M_revenue_previous_month =
    CALCULATE(
        [M_Revenue_order],
        DATEADD(dim_calendar[Date], -1, MONTH)
        )
```

Time Intelligence Practice.

How to make some advanced metrics?

Year	Month	revenue	M_revenue_YTD	M_revenue_YTD_2	M_revenue_YTD_3	M_total_revenue_year	M_%_revenue_in_total_year	M_revenue_lastmonth	M_revenue_previous_month	M_revenue_MOM
2023	9	0.48M	481,961.79	481,961.79	481,961.79	6,849,562.61	7.04%			0.0%
2023	10	2.32M	2,800,428.14	2,800,428.14	2,800,428.14	6,849,562.61	33.85%	481,961.79	481,961.79	381.0%
2023	11	2.07M	4,868,862.28	4,868,862.28	4,868,862.28	6,849,562.61	30.20%	2,318,466.35	2,318,466.35	-10.8%
2023	12	1.98M	6,849,562.61	6,849,562.61	6,849,562.61	6,849,562.61	28.92%	2,068,434.14	2,068,434.14	-4.2%
2024	1	6.62M	6,620,343.20	6,620,343.20	13,469,905.81	56,753,276.49	11.67%	1,980,700.33	1,980,700.33	234.2%
2024	2	5.73M	12,354,039.26	12,354,039.26	19,203,601.87	56,753,276.49	10.10%	6,620,343.20	6,620,343.20	-13.4%
2024	3	6.32M	18,678,407.10	18,678,407.10	25,527,969.71	56,753,276.49	11.14%	5,733,696.06	5,733,696.06	10.3%
2024	4	6.42M	25,096,660.72	25,096,660.72	31,946,223.33	56,753,276.49	11.31%	6,324,367.84	6,324,367.84	1.5%
2024	5	6.71M	31,805,703.65	31,805,703.65	38,655,266.26	56,753,276.49	11.82%	6,418,253.62	6,418,253.62	4.5%
2024	6	6.67M	38,474,337.24	38,474,337.24	45,323,899.85	56,753,276.49	11.75%	6,709,042.93	6,709,042.93	-0.6%
2024	7	6.54M	45,009,466.76	45,009,466.76	51,859,029.37	56,753,276.49	11.51%	6,668,633.59	6,668,633.59	-2.0%
2024	8	6.71M	51,715,585.37	51,715,585.37	58,565,147.98	56,753,276.49	11.82%	6,535,129.52	6,535,129.52	2.6%
2024	9	5.04M	56,753,276.49	56,753,276.49	63,602,839.10	56,753,276.49	8.88%	6,706,118.61	6,706,118.61	-24.9%
Total		**63.60M**	**56,753,276.49**	**56,753,276.49**	**63,602,839.10**	**63,602,839.10**	**100.00%**		**58,565,147.98**	**0.0%**

```
M_revenue_YTD =
    CALCULATE(
        [M_Revenue_order],
        DATESYTD(dim_calendar[Date])
        )
```

```
M_revenue_YTD_2 =
    TOTALYTD(
        [M_Revenue_order],
        dim_calendar[Date]
        )
```

```
M_revenue_YTD_3 =
    CALCULATE([M_Revenue_order],
    FILTER(
        ALL(fact_order),
        fact_order[purchase date] <= MAX(fact_order[purchase date])
        )
    )
```

Time Intelligence Practice.

How to make some advanced metrics?

Year	Month	revenue	M_revenue_YTD	M_revenue_YTD_2	M_revenue_YTD_3	M_total_revenue_year	M_%_revenue_in_total_year	M_revenue_lastmonth	M_revenue_previous_month	M_revenue_MOM
2023	9	0.48M	481,961.79	481,961.79	481,961.79	6,849,562.61	7.04%			0.0%
2023	10	2.32M	2,800,428.14	2,800,428.14	2,800,428.14	6,849,562.61	33.85%	481,961.79	481,961.79	381.0%
2023	11	2.07M	4,868,862.28	4,868,862.28	4,868,862.28	6,849,562.61	30.20%	2,318,466.35	2,318,466.35	-10.8%
2023	12	1.98M	6,849,562.61	6,849,562.61	6,849,562.61	6,849,562.61	28.92%	2,068,434.14	2,068,434.14	-4.2%
2024	1	6.62M	6,620,343.20	6,620,343.20	13,469,905.81	56,753,276.49	11.67%	1,980,700.33	1,980,700.33	234.2%
2024	2	5.73M	12,354,039.26	12,354,039.26	19,203,601.87	56,753,276.49	10.10%	6,620,343.20	6,620,343.20	-13.4%
2024	3	6.32M	18,678,407.10	18,678,407.10	25,527,969.71	56,753,276.49	11.14%	5,733,696.06	5,733,696.06	10.3%
2024	4	6.42M	25,096,660.72	25,096,660.72	31,946,223.33	56,753,276.49	11.31%	6,324,367.84	6,324,367.84	1.5%
2024	5	6.71M	31,805,703.65	31,805,703.65	38,655,266.26	56,753,276.49	11.82%	6,418,253.62	6,418,253.62	4.5%
2024	6	6.67M	38,474,337.24	38,474,337.24	45,323,899.85	56,753,276.49	11.75%	6,709,042.93	6,709,042.93	-0.6%
2024	7	6.54M	45,009,466.76	45,009,466.76	51,859,029.37	56,753,276.49	11.51%	6,668,633.59	6,668,633.59	-2.0%
2024	8	6.71M	51,715,585.37	51,715,585.37	58,565,147.98	56,753,276.49	11.82%	6,535,129.52	6,535,129.52	2.6%
2024	9	5.04M	56,753,276.49	56,753,276.49	63,602,839.10	56,753,276.49	8.88%	6,706,118.61	6,706,118.61	-24.9%
Total		**63.60M**	**56,753,276.49**	**56,753,276.49**	**63,602,839.10**	**63,602,839.10**	**100.00%**		**58,565,147.98**	**0.0%**

```
M_revenue_MOM =
    IF(
        ISBLANK(
            [M_revenue_lastmonth]),
            0,
            ([M_Revenue_order] - [M_revenue_lastmonth]) / [M_revenue_lastmonth]
    )
```

Card visualization.

How to make some advanced card?